Day 1/100
Embedded Firmware

Topics covered for the day:

I. Basic Pointers

1. Meaning, Uses,
2. Most used Syntax and its meaning

II. Functions and Arrays

1. Both integrated in pointers through codes.

III. Pointer + functions + array + codes:

1. Sum and product of 2 elements in an array
2. Square and cube of a number using pointers
3. Prime number detection using basic functions
4. Delete duplicates in an array using pointers
5. Sort and print array using pointer

Functions

- In C, a Function performs a specific task.
- Instead of writing the same logic multiple times, we will write logic only once and call the function to perform the same task whenever needed.
- Function will have:

```
return_type function_name(parameters){
    // body
}
```

- Ex: Here, **square()'s** job is to just return the square of n.

```
#include <stdio.h>

int square(int n) {
    return n * n;
}

int main() {
    int num = 0;
    scanf("%d",&num);
    int result = square(num);
    printf("%d", result);
    return 0;
}
```

Input	Output
5	25

Pointers

- Pointer is a variable that stores the address of other variable.
- Used to access and/or modify the original data using its base address.
- Used mainly in the functions, arrays, strings, dynamic memory allocation, Linked List etc.

Syntax	Meaning
`&`	Address operator
`*`	Dereference operator
`**`	Pointer to pointer
`int *ptr = &a;`	ptr stores address of a
`printf("%d", *ptr);`	**Prints** value pointed by ptr (**dereferencing**)
`printf("%p", ptr);`	**Prints** address stored in p
`*ptr = 25;`	**Changes** value of a through pointer
`ptr = &b;`	Now ptr points to varible b
`int **pptr = &ptr;`	pptr **stores address of existing** ptr (pointer to pointer)
`*q = 50;`	Changes value of b through q
`ptr = q;`	Both ptr and q **point to b**
`int arr[3] = {1,2,3};` `int *ptr = arr;`	ptr points to first array element
`*(ptr+1); or *(ptr+2);`	Accesses **next array element** through indexing

Pointers (cont.)

- Common ways to pass data in functions:
  1. Pass by value
  2. Pass by reference

Concept	Meaning
Pass by value	Function gets a copy of variable
Pass by reference	Function gets address of variable
Original value in pass by value	Does not change
Original value in pass by reference	Can change
If having small data	Use Pass by value
Want to modifying data?	Use Pass by reference

Arrays

- It is a collection of elements arranged in the continuous memory locations
- Array index elements start from 0.
- Reading and writing is done using loops.

Syntax	Meaning
`int arr[5];`	Declares an integer array of size 5
`int arr[5] = {1,2,3,4,5};`	Declares and initializes array
`arr[0]`	Read First element
`arr[4]`	Read Fifth element

`For(int i=0; i<10; i++){`
`  printf/scanf(“%d”,& arr[i]);`
`}`
	Loop through the elements. Read / Write elements

`arr[2] = 10;`	Updates third element
`printf("%d", arr[0]);`	Prints first element
`scanf("%d", &arr[0]);`	Reads value into first element

1. Sum and product of 2 elements in using pointers

```c
// FIND SUM AND PROD OF 2 NOS USING POINTERS
#include<stdio.h>
static int sum=0, prod=0;
int sum_and_prod(int*,int*);
int main(){
    int num1=0, num2=0;
    printf("Enter the number 1 : ");
    scanf("%d",&num1);
    printf("Enter the number 2 : ");
    scanf("%d",&num2);
    sum_and_prod(&num1, &num2);

}
int sum_and_prod(int*ptr1, int*ptr2){
    sum=*ptr1+*ptr2;
    printf("Sum is %d\n",sum);
    prod=(*ptr1) * (*ptr2);
    printf("Product is %d\n",prod);
}
```

Input	Enter the number 1 : 2 Enter the number 2 : 3
Output	Sum is 5 Product is 6

2. Square and cube of a number using pointers

```c
// PROG FOR FINDING SQUARE AND CUBE OF A NUM;
#include<stdio.h>
void sq(int num, int *square);
void cb(int num, int *cube);
int main(){
    int num=0, cube=0, square=0;
    scanf("%d",&num);
    sq(num,&square);
    cb(num,&cube);
    printf("Square is %d\nCube is %d\n",square,cube);
}

void sq(int num,int*square){
    *square=num * num;
}
void cb(int num, int *cube){
    *cube=num*num*num;
}
```

Input	5
Output	Square is 25 Cube is 125

3. Prime number detection using basic functions

```c
// CHECK IF NUM IS PRIME OR NOT USING FUNCTION
#include <stdio.h>
int is_prime(int num){
    int i;
    //check
    if (num <= 1){
        return 0;
    }
    // divisibility
    for (i = 2; i * i <= num; i++){
        if (num % i == 0){
            return 0;   // Not prime
        }
    }
    return 1;   // Prime
}
```

```c
int main(){
    int num, result;
    printf("Enter a number: ");
    scanf("%d", &num);


    /* Invalid input check */
    if (num < 0){
        printf("Invalid input\n");
        return 0;
    }


    result = is_prime(num);


    if (result == 1){
        printf("%d is a prime number\n", num);
    }
    else{
        printf("%d is not a prime number\n", num);
    }
    return 0;
}
```

Input

Output

```
Enter a number: 5
5 is a prime number
```

Input

Output

```
Enter a number: 6
6 is not a prime number
```

4. Delete duplicates in an array using pointers

```c
//delete duplicates
#include <stdio.h>

void remove_duplicates(int arr[], int size,
int uniq[], int *uniq_size);


int main(){
    int size=0, uniq_size=1;
    scanf("%d",&size);
    int arr[size];

    int uniq[size];
    //write the elements size times.
    for(int i=0; i<size; i++){
        scanf("%d",&arr[i]);
    }
    //call function
    remove_duplicates(arr, size, uniq,
&uniq_size);
    return 0;
}
```

```c
void remove_duplicates(int arr[], int size, int
uniq[], int *uniq_size){

    *uniq_size=1;
    uniq[0]=arr[0];

        for(int i=1; i<size; i++){
        int is_duplicate=0;
        for(int j=0; j<*uniq_size; j++){
            if (arr[i]==uniq[j]){
                is_duplicate=1;
                break;
            }
        } if(!is_duplicate){
            uniq[*uniq_size]=arr[i];
            (*uniq_size)++;
        }
    }
    //print the uniq elements:
    printf("After removing duplicates: ");
    for(int i=0; i<*uniq_size; i++){
        printf("%d ",uniq[i]);
    }
}
```

Input	Enter array size : 5 Enter array elements : 1 2 2 3 3
Output	After removing duplicates: 1 2 3

5. Sort and print array using pointer

```c
#include <stdio.h>

void print_sorted(int arr[], int size);

int main() {
    int size = 0;
    printf("Enter the array size: ");
    if (scanf("%d", &size) != 1 || size <= 0) {
        printf("Invalid size.\n");
        return 1;
    }

    int arr[size];
    printf("Enter %d elements: ", size);
    for (int i = 0; i < size; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Sorted output: ");
    print_sorted(arr, size);
    printf("\n");

    return 0;
}
```

```c
void print_sorted(int arr[], int size) {
    int visited[size];
// Initialize visited array to 0 (false)
    for (int i = 0; i < size; i++) {
        visited[i] = 0;
    }

 // Outer loop to print size number of elements
    for (int count = 0; count < size; count++) {
        int min_idx = -1;


 // Find the smallest element that hasn't been visited yet
        for (int i = 0; i < size; i++) {
            if (!visited[i]) {
                if (min_idx == -1 || arr[i] < arr[min_idx])
                    min_idx = i;
                }
            }
        }


// Print the smallest found and mark it
        if (min_idx != -1) {
            printf("%d ", arr[min_idx]);
            visited[min_idx] = 1;
        }
    }
}
```

Input	Enter the array size: 5 Enter 5 elements: 55 42 12 6 99
Output	Sorted output: 6 12 42 55 99